



STINK 3014 (NEURAL NETWORKS) A251 – Assignment (#1) (10 %)

Instructor: Azizi Ab Aziz, *PhD*

Submission date: 3rd Nov 2025 (before 11.59 pm) via UUM Learning Portal

“Raise your words, not voice. It is rain that grows flowers, not thunder.” [Rumi]

PART A: THEORETICAL CONCEPTS

Topic #1: Introduction to Neural Networks

1. Describe the actual process of our biological neuron and how it has inspired the development of artificial neural networks.
2. Read the following scenarios
“A neuroscientist argues that artificial neural networks should incorporate temporal spikes and neurotransmitter dynamics, while an AI engineer claims current mathematical models already capture sufficient functionality”
 - a) Explain what each side (argument) means.
 - b) Discuss which aspects of brain functioning might improve artificial neural networks if implemented, **and** which are unnecessary for efficient computation.
3. Discuss why ANNs require large datasets and many computational loops (iterations/ epochs) whereas biological learning can happen with few examples.
4. Discuss FIVE (5) main advantages and disadvantages of using neural network applications in a real-world context.

Topic #2: Linear Models and Perceptron

1. Explain how Hebbian learning differs from Perceptron learning in:
 - a) Use of target outputs
 - b) Update rule
 - c) Supervised vs unsupervised nature
 - d) Would the same dataset produce the same final weights under Perceptron learning?
2. This question requires the execution of a given Python code (related to our previous hands-on example) (file: **STINK3014_Assignment01_Driving_Satisfaction_Hebbian_Model.py**). Based on this file, you are required to answer the following questions with experimental results (by varying related parameters as needed for related questions):
 - a) Without changing any parameters, discuss which features are more strongly correlated with driver satisfaction.
 - b) Discuss the role of the learning (η) in this code. Based on your experiments, what would happen if η were too small (e.g., 0.001) or too large (e.g., 0.99)?
 - c) The initial weights are all set to 0.33. Discuss how different initial weights might affect the learning trajectory. Would the final weight pattern be the same? Why or why not?
 - d) Explain the real-world application of these weights (related to the driving satisfaction scenario).

3. What is meant by an activation function in artificial neurons?
4. You are given the following binary classification dataset representing the logical AND function: with four input-output pairs (x_1, x_2, a) as [#1(0,0,0), #2(0,1,0), #3(1,0,0), #3(1,1,1)]. The perceptron model computes the outputs as:

$$y = \begin{cases} 1, & \sum_{i=1}^2 x_i w_i + b w_b \geq 0 \\ 0, & x < 0 \end{cases}$$

And updates its weights according to the Perceptron Learning Rule (Delta Rules)

$$w_{(i,t+1)} = w_{(i,t)} + \eta \cdot (a_i - y_i) \cdot x_i$$

$$w_{(b,t+1)} = w_{(b,t)} + \eta \cdot (a_i - y_i)$$

Where $w_1=0.2$, $w_2 = 0.2$, $b = 1$, $w_b = -0.1$ and learning rate $\eta = 0.1$. Using this information, show computational / calculation steps when you are training the perceptron for two epochs using the logical AND function dataset for THREE (3) epochs.

- a) Does the perceptron correctly classify all input patterns?
- b) How does the bias term affect the decision boundary?
5. Why is zero initialisation of weight not a good practice?
6. Explain why linear separability is a necessary condition for the single-layer perceptron is to converge.

Topic #3: Multilayer Perceptron

1. Explain the main goal of the backpropagation algorithm in training a multilayer perceptron.
2. Why is the differentiability of the activation function crucial in MLPs?
3. Define hidden layers, optimisers, epoch, batch size, and learning rate in the context of MLP training. Explain how varying each parameter affects:
 - a) Convergence speed
 - b) Stability of learning
 - c) Model generalisation
4. Explain the purpose of a loss function in backpropagation with respect to weight functions.
5. Compare Mean Squared Error (MSE) and Cross-Entropy Loss in the context of MLPs. Which loss function is more suitable for regression problems/classification problems

PART B: PRACTICAL IMPLEMENTATION (PYTHON)

**This part requires you to solve the given questions using the Python programming language*



Consider a simple and primitive small animal neuron, which controls the behaviour that can be defined by its instinct to collect small red berries and avoid natural predators. From previous experiences, these animals have been conditioned to approach an object under three conditions:

- the size is small and round-shaped
- the colour is red
- No local movement is detected

Note that local movement indicates to the animal that a predator may be present. This behaviour is reflected in the table below (yes = 1 and no = 0)

Small	Red	Movement	Approach
0	1	1	0
0	1	0	0
1	0	0	1
1	1	0	1

This example neuron employs three inputs (x), which are identified by the characteristics on which they are dependent: small (x_1), red (x_2), and movement (x_3). The output (y) defines whether the animal will ($y = 1$) or will not ($y = 0$) approach an object.

Instruction:

As an AI scientist, you are required to build a brain-inspired Perceptron model-based learning technique (based on the application of the error-correction learning rule and supervised perceptron algorithm) that is utilised to define appropriate synapse weights in Python programming language. Based on your developed simulator, predict (either approach = yes/no) the outcome of these new cases:

Small	Red	Movement	Approach
1	1	1	?
1	0	1	?
0	0	0	?

Your report will provide three deliverables:

- a) Include how you came up with the values of α , epochs, initial conditions of w_1 , w_2 , w_3 and the bias used.
- b) Graph: illustrating how your method converged and how the prevalence of error decreased with iteration. Clearly label all data sets and axis labels.
- c) Python Code: You will include your Python code as an Appendix. Please include your code in a single file so that I may run it myself (rename your code as **<your_matric_num>_A251_A01.py**)

Policy:

All grading of deliverables will be based on the standards indicated for each deliverable. Deliverables may not be turned in late, and no cheating! For this class, cheating will include: plagiarism (using the writings of another without proper citation), copying of another (either current or past student's work), working with another on individually assigned work, or in any other way presenting as one's work that which is not entirely one's work. The occurrence of plagiarism will result in removal from the course with a failing grade.